

ORDER SENSITIVITY AND ADAPTATION UTILITY IN A CONTROLLED TEST-TIME ADAPTATION MODEL*

Anonymous authors

Paper under double-blind review

ABSTRACT

Test-time adaptation (TTA) updates a model online, so its outcome can depend on the order in which the test stream arrives. We ask a deliberately narrow question in a controlled synthetic setting: when meta supervision, adaptation budget and batch membership are held fixed, does an order penalty add utility? The model is a linear softmax classifier whose 4-dimensional input affine transform (2×4 parameters) is adapted by plain-SGD entropy minimization; we call this an entropy-minimization toy baseline rather than Tent. The candidate restricts adaptation to a subspace chosen from a discrete pool by trading a supervised utility estimate on labelled meta data against the local second-order order-difference term of plain SGD. Controls include a utility-only subspace that shares the candidate’s meta data, labels, statistics and rank but drops the penalty, two reduced learning rates, and two update-norm-matched controls, one of which is causal per step. Over 3 synthetic world seeds and 8 fixed orders each, with batch membership fixed and only batch order permuted, a pre-registered rule returned *order penalty added utility not supported*. The candidate’s gain over no adaptation was +0.29, -3.10 and -7.42 percentage points in the three seeds. In two seeds the candidate and the utility-only control selected the same projector and learning rate and produced identical trajectories; in the third the candidate was worse. The entropy baseline itself did not improve on no adaptation here, so this environment cannot adequately evaluate a gain-retention hypothesis. The results concern this synthetic implementation only and carry no conclusion about image TTA or order regularization in general.

1 INTRODUCTION

Fully test-time adaptation updates a pretrained model on unlabelled test batches as they arrive, for example by minimizing prediction entropy over normalization parameters (Wang et al., 2021). Because every update depends on the parameters left by the previous one, the final model is a function of the order of the stream. Order dependence is not new: temporally correlated or non-i.i.d. streams degrade several TTA methods (Gong et al., 2022; Yuan et al., 2023; Niu et al., 2023; Zhao et al., 2023), reordering alone changes results in continual settings (Wang et al., 2022; Döbler et al., 2023; Gao et al., 2023), and resetting schemes are used to contain long-horizon drift (Press et al., 2023; Niu et al., 2023; Lim et al., 2026; Hoang et al., 2024). For plain SGD the leading order-dependent term is also known: two consecutive steps on losses L_a and L_b differ from the reversed order by $\eta^2(H_b g_a - H_a g_b)$ up to $O(\eta^3)$, a consequence of Taylor expansion that appears in the analyses of Reptile, of implicit regularization in SGD and of data-ordering attacks (Nichol et al., 2018; Smith et al., 2021; Shumailov et al., 2021), and more recently as a predictor of curriculum order (Sweeney, 2026).

A natural idea is therefore to *penalize* that term: restrict adaptation to directions along which the local order difference, measured on model outputs, is small. Whether such a penalty is useful is an empirical question, and it is easy to answer it wrongly. A restricted subspace moves less, uses extra supervision if it is fitted on labelled data, and can change batch composition effects when orders are formed by reshuffling samples. Any of these can reduce order sensitivity without the penalty playing a role. This paper reports a controlled diagnostic of exactly this confound structure.

*Internal working draft v0. Not submitted to, and not under review at, any venue. The “Paper under double-blind review” line is printed by the official anonymous template and does not describe this draft.

Question. With the same meta supervision, the same adaptation budget and the same batch membership, does an order penalty add utility over (i) a utility-only subspace that differs from the candidate only by the penalty, (ii) simply using a smaller learning rate, and (iii) matching the size of each update?

What we do. We build a small synthetic TTA model in which every relevant quantity can be controlled and audited (Section 3): a linear softmax classifier with an adapted input affine transform, a stream replayed in fixed orders whose batch membership is fixed so that only batch order varies, a common terminal holdout for stationary streams, and current-regime holdouts plus predict-then-update online error for genuine drift. The candidate and all controls are tuned on the same development split with the same grid. The decision rule, seeds and minimum effects of interest were fixed and committed before the run, and every reported number is exported programmatically from the raw run files.

What we find. The pre-registered rule returned *order penalty added utility not supported* (Section 5). In two of three world seeds the penalized and the utility-only fitters selected the same two pool directions and the same learning rate, which makes their trajectories identical by construction; in the third seed the two differed and the candidate was worse. The entropy-minimization baseline did not improve on no adaptation in this environment, which limits what the environment can say about retaining adaptation gain.

Scope of the contribution. We do not claim new theory: the order-difference expansion is standard and we restate it only to define the candidate (Section 4). We do not claim evidence about real-image TTA; no real data were used. The contribution is a controlled protocol, with its controls and audits, and a negative diagnostic result reported together with the implementation-level reason for it.

2 RELATED WORK

Test-time adaptation and stream order. Tent adapts batch-normalization affine parameters by entropy minimization while re-estimating normalization statistics on each test batch (Wang et al., 2021). EATA filters samples by entropy and adds a Fisher regularizer (Niu et al., 2022). SAR targets mixed shifts, single-sample batches and online label shift, the last of which is constructed by reordering samples, and resets the model when an entropy statistic collapses (Niu et al., 2023). NOTE and RoTTA study temporally correlated streams (Gong et al., 2022; Yuan et al., 2023), and TTAB documents how online batch dependency complicates model selection (Zhao et al., 2023). CoTTA reports variation across corruption orders and uses stochastic restoration (Wang et al., 2022); RMT reports a large gap between easy-to-hard and hard-to-easy orders (Döbler et al., 2023); DDA reports that Tent is sensitive to shuffled versus ordered data (Gao et al., 2023). Our work does not propose a TTA method for these benchmarks. It isolates one proposed mechanism for reducing order sensitivity in a setting small enough to control.

Resetting and continual TTA. RDumb resets periodically (Press et al., 2023), PeTTA analyses collapse in recurring scenarios (Hoang et al., 2024), and ASR decides adaptively when and where to reset (Lim et al., 2026). Resetting bounds how far adaptation can drift; it is complementary to, and not tested in, our short-horizon study.

Order terms of SGD. The second-order expansion of sequential SGD steps underlies the gradient-alignment interpretation of Reptile and Fish (Nichol et al., 2018; Shi et al., 2022), the implicit regularizer of SGD with fixed permutations and its average over orders (Smith et al., 2021), and data-ordering attacks (Shumailov et al., 2021). Sweeney (2026) uses the corresponding commutator to choose the order of source domains. We use the same term for a different purpose, to select an adaptation subspace, and we test that use against controls rather than proposing it as a finding.

Low-dimensional and meta-learned adaptation. Training and adaptation can often be confined to low-dimensional parameter subspaces (Li et al., 2018; Gur-Ari et al., 2018); Tent’s normalization parameters are themselves a small subset. Meta-learned TTA such as MT3 uses supervised source

data at meta-training time (Bartler et al., 2022). Our candidate and utility-only control likewise use labelled meta data, so neither is fully source-free.

3 PROBLEM AND EXPERIMENTAL SETTING

Synthetic worlds. Each world seed generates 3 Gaussian classes in $\mathbb{R}^4$ and several shifted domains, each an independent per-feature affine map of the source distribution plus small noise. Disjoint splits are drawn from the same seed: labelled source data (to train the classifier), meta data under 4 simulated shifts (used only to fit subspaces), development and calibration streams with a development holdout under 2 validation shifts, and a test stream and test holdout under 4 test shifts. The test stream has 480 samples and the test holdout 600. No test label is visible to any adaptation arm (Appendix D).

Model and entropy-minimization toy baseline. A linear softmax classifier $f(x) = Wz + b$ is trained on source data and frozen. Test-time adaptation updates $\theta = (\gamma, \beta) \in \mathbb{R}^{2 \times 4}$ in $z = \gamma \odot \hat{x} + \beta$, where $\hat{x}$ is standardized with *fixed source statistics*; there are no running buffers and no optimizer state. Each step takes one plain-SGD step, with constant learning rate η , on the mean prediction entropy of the current batch. We call this the *entropy-minimization toy baseline* (EM-toy). It shares Tent’s objective but differs from the published Tent configuration, which adapts normalization affine parameters of a deep network and re-estimates normalization statistics on each test batch (Wang et al., 2021). Run identifiers keep the historical arm id `tent_full`; the mapping to display names is in `paper/tables/display_names.tsv`.

Orders and conditions. One multiset of test sample IDs is replayed in 8 fixed orders per world seed, and every arm sees the same orders. In the *primary stationary* condition the stream is partitioned once into batches of 16 and orders permute only the batches (30 steps), so batch composition is identical across orders. The *secondary* condition reshuffles samples and re-forms batches for every order; it is reported separately and not used for the decision. In the *drift* condition each test domain is partitioned once into single-domain batches and orders permute the sequence of domain blocks and the batches within each block (32 steps). Before every run the harness checks that the adapter starts from the source parameters and $\theta_0 = (\mathbf{1}, \mathbf{0})$, that frozen parameters are unchanged afterwards, and that the global random state was not consumed.

Metrics. For the stationary conditions the primary metric is the terminal error on a holdout that is shared by all orders and arms. For drift we do not score the mixture holdout, which would penalise adaptation to the latest regime. Instead we score the current regime’s holdout at the end of every domain block and average over blocks, and we report predict-then-update online error separately. For each world seed and arm we report the mean over the observed orders, the standard deviation (SD) across them, the worst observed order and the gain over no adaptation. Orders are nested in world seeds. We therefore report per-seed values and paired per-seed differences (candidate minus control over the same orders), and we form no confidence interval that pools seeds and orders. With 3 seeds, the independent units are the seeds.

Arms and information access. All adapting arms use the same loss, the same number of steps and no test labels.

- *No adaptation*: the frozen source model.
- *EM-toy (full affine)*: all 2×4 parameters, with a tuned η .
- *EM-toy, $lr \times 0.3$ and $lr \times 0.1$* : small-step controls.
- *Order-penalized subspace (candidate)*: Section 4; it uses labelled meta data at fit time.
- *Utility-only subspace*: the same meta batches, meta labels, candidate pool and second-order statistics as the candidate, restricted to the candidate’s realised rank, with the order penalty set to zero.
- *Per-step norm-matched (causal)*: full-space direction, with step t rescaled to the norm of step t of a lockstep replica of the candidate, so no future step is readable.

- *Global norm-matched*: a full-space η bisected on the calibration stream so that the mean terminal displacement matches the candidate's; this reads no labels.

The EM-toy, candidate and utility-only arms select η from the same grid $\{0.05, 0.1, 0.2, 0.5, 1, 2\}$ on development streams, by development holdout error (mean current-regime error for drift). Ties go to the smaller value. The grid was not widened for this study.

Pre-registered decision rule. The rule, the seeds (the first three of five pre-registered world seeds and the first eight of sixteen orders) and the minimum effects of interest (MIE; gain 1.00 pp, SD 0.50 pp, tolerance 0.50 pp) were committed before the run (commit 0119a34). The candidate is flagged for a later real-data pilot only if all of the following hold:

1. In the primary stationary condition, its gain over no adaptation is at least the MIE on average and positive in every seed.
2. Its SD reduction relative to EM-toy is at least the MIE on average and positive in every seed.
3. It *dominates* each required control. That is, it is better by the MIE in mean error, or in SD, with the same sign in every seed, and not worse than the tolerance on the other axis.
4. In drift, its current-regime and online errors are within tolerance of the utility-only control and not worse than no adaptation.

A required cell that failed, hit a cap or did not run makes the result *incomplete*; otherwise the result is *not supported*.

4 METHOD UNDER STUDY

Order difference of two plain-SGD steps (standard). Let $U_a(\theta) = \theta - \eta \nabla L_a(\theta)$ and write $g_a = \nabla L_a(\theta)$ and $H_a = \nabla^2 L_a(\theta)$ at the common starting point θ . Applying a then b gives

$$U_b(U_a(\theta)) = \theta - \eta g_a - \eta \nabla L_b(\theta - \eta g_a). \quad (1)$$

If $\nabla^2 L_b$ is M_b -Lipschitz on the segment $[\theta, \theta - \eta g_a]$, Taylor's theorem gives $\nabla L_b(\theta - \eta g_a) = g_b - \eta H_b g_a + \rho_b$ with $\|\rho_b\| \leq \frac{M_b}{2} \eta^2 \|g_a\|^2$. The symmetric statement holds for ρ_a on $[\theta, \theta - \eta g_b]$. Subtracting the reversed order,

$$U_b(U_a(\theta)) - U_a(U_b(\theta)) = \eta^2 (H_b g_a - H_a g_b) - \eta (\rho_b - \rho_a), \quad (2)$$

$$\|\eta (\rho_b - \rho_a)\| \leq \frac{\eta^3}{2} (M_b \|g_a\|^2 + M_a \|g_b\|^2). \quad (3)$$

For quadratic losses the Hessians are constant ($M_a = M_b = 0$), so the second-order term is the *exact* two-step difference for every η . For smooth non-quadratic losses it is only the leading term, and Eqs. (2)–(3) are informative only when η times the local curvature is small. Our numerical fixtures show a relative remainder above 0.1 at $\eta L \approx 0.3$ (Appendix C). Eq. (2) is a restatement of a known expansion (Nichol et al., 2018; Smith et al., 2021; Sweeney, 2026). It says nothing about adaptive optimizers, long-horizon stability or generalization, and we do not use it as a guarantee. For K steps in order π the analogous leading term is $\eta^2 \sum_{i < j} H_{\pi_j} g_{\pi_i}$; for $K \geq 3$ it has cubic corrections even for quadratics.

Candidate: order-penalized adaptation subspace. For an orthonormal P_S spanning a subspace S , projected SGD replaces g by $P_S P_S^\top g$. The candidate chooses S at θ_0 from a discrete pool of 2×4 orthonormal directions. The pool consists of the eigenvectors of the uncentred second moment of meta-batch entropy gradients, completed by coordinate axes. On 8 single-domain meta batches a we compute the entropy gradient g_a , the supervised cross-entropy gradient s_a (this requires meta labels), entropy Hessian-vector products $H_a p_j$, and output Jacobian-vector products $J p_j$ of the class probabilities on 96 meta probe points, all by finite differences where needed. The fitter scores

$$\text{score}(S) = \frac{\text{mean}_a \langle P_S s_a, P_S g_a \rangle}{G_{\text{ref}}} - \lambda \sqrt{\frac{\text{mean}_{a \neq b} \|J P_S (H_b P_S g_a - H_a P_S g_b)\|^2}{\text{mean}_{a \neq b} \|J (H_b g_a - H_a g_b)\|^2}}, \quad (4)$$

Table 1: Primary stationary condition (fixed batch membership, batch order permuted). Err: mean terminal common-holdout error over the 8 observed orders (%); SD: standard deviation over those orders (percentage points, pp); Gain: no-adaptation error minus arm error (pp; positive is better). Orders are nested in world seeds and are not independent replicates.

Arm	World seed r0			World seed r1			World seed r2		
	Err	SD	Gain	Err	SD	Gain	Err	SD	Gain
No adaptation	20.83	0.00	+0.00	18.17	0.00	+0.00	36.83	0.00	+0.00
EM-toy (full affine)	23.25	0.84	−2.42	19.65	2.40	−1.48	36.98	0.11	−0.15
EM-toy, lr $\times 0.3$	22.79	0.32	−1.96	17.94	0.38	+0.23	36.35	0.06	+0.48
EM-toy, lr $\times 0.1$	22.00	0.13	−1.17	17.67	0.09	+0.50	36.46	0.08	+0.38
Order-penalized subspace (candidate)	20.54	0.08	+0.29	21.27	0.56	−3.10	44.25	1.21	−7.42
Utility-only subspace	20.54	0.08	+0.29	17.67	0.00	+0.50	44.25	1.21	−7.42
Per-step norm-matched (causal)	20.83	0.00	+0.00	17.96	2.84	+0.21	39.02	1.18	−2.19
Global norm-matched (calibrated)	20.88	0.08	−0.04	20.08	3.18	−1.92	36.35	0.06	+0.48

where $G_{\text{ref}} = \sum_j \max(0, \text{gain}(\{p_j\}))$ is the largest first-order supervised gain any pool subset attains. The first term is the retained fraction of first-order usefulness. The second is λ times the retained fraction of the output-level second-order order term of Eq. (2), restricted to S . Directions are added greedily up to $k_{\text{max}} = 4$, and only while they increase the score; this early stop sets the candidate’s rank. We fixed $\lambda = 1$ and $k_{\text{max}} = 4$ in advance and did not tune them.

Utility-only ablation. The utility-only control reuses the same pool, meta batches, labels and statistics, sets $\lambda = 0$, and takes exactly as many directions as the candidate selected. Because the utility term is additive over orthonormal pool directions, this is the top- k set by per-direction supervised gain. The two arms therefore differ only through the penalty’s effect on *which* directions are selected. When they select the same directions and the development split selects the same η , the two runs are the same deterministic computation and must coincide.

5 EXPERIMENTS AND OBSERVATIONS

All numbers below come from a single recorded run of the frozen configuration at commit 0119a34. That run covers 3 conditions $\times$ 3 world seeds $\times$ 8 orders $\times$ 8 arms, and 576 of its 576 cells completed. The cells are not independent replicates: orders are nested in world seeds, and the three conditions reuse the same worlds. The whole run used 128.9 s of a 3,600 s CPU budget, with peak resident memory 79.7 MiB, on one worker and one thread.

Decision. The pre-registered rule returned *order penalty added utility not supported*, with 9 failed conditions. The candidate’s stationary gain over no adaptation was +0.29, −3.10 and −7.42 pp in the three seeds, so the gain condition failed. Its SD reduction relative to EM-toy was +0.76, +1.84 and −1.10 pp, negative in one seed, so the robustness condition failed. It dominated none of the five required controls. In drift its seed-mean gain over no adaptation was −3.35 pp (current regime) and −3.59 pp (online).

Primary stationary condition (Table 1). With fixed batch membership, EM-toy did not improve on no adaptation in any seed: its gains were −2.42, −1.48 and −0.15 pp. Its order SD was 0.84, 2.40 and 0.11 pp. The small-step controls had a smaller SD than EM-toy in every seed. For lr $\times$ 0.1 the gains were −1.17, +0.50 and +0.38 pp, all below the MIE. The candidate had the lowest error in seed r0, tied with the identical utility-only run discussed below, but it was worse than no adaptation in seeds r1 and r2. In seed r2 its error was 44.25% against 36.83% without adaptation. In this environment low order sensitivity is attained most easily by adapting little: no adaptation has zero SD by construction.

Paired comparison with the controls (Table 2). Relative to the utility-only control, the candidate’s mean-error difference was +0.00, +3.60 and +0.00 pp. Relative to the reduced-learning-rate and norm-matched controls it was better in seed r0 and worse in seeds r1 and r2. Under the rule, a

Table 2: Paired differences, candidate minus control, computed per world seed over the same 8 orders (pp; negative favours the candidate). ΔErr : mean terminal error difference; ΔSD : difference of across-order standard deviations. Primary stationary condition.

Control	World seed r0		World seed r1		World seed r2	
	ΔErr	ΔSD	ΔErr	ΔSD	ΔErr	ΔSD
Utility-only subspace	+0.00	+0.00	+3.60	+0.56	+0.00	+0.00
EM-toy, lr $\times 0.3$	-2.25	-0.24	+3.33	+0.18	+7.90	+1.15
EM-toy, lr $\times 0.1$	-1.46	-0.05	+3.60	+0.47	+7.79	+1.13
Per-step norm-matched (causal)	-0.29	+0.08	+3.31	-2.28	+5.23	+0.03
Global norm-matched (calibrated)	-0.33	+0.00	+1.19	-2.63	+7.90	+1.15
EM-toy (full affine)	-2.71	-0.76	+1.62	-1.84	+7.27	+1.10

Table 3: Candidate (order-penalized) versus utility-only subspace per condition and world seed. Pool indices of the selected directions (both fitters select from the same shared pool and meta statistics), tuned learning rates, and whether the terminal parameters (SHA-256 of θ_T) and every per-step update norm coincide on all 8 orders, read from the raw logs.

Condition	Seed	Cand. idx	Util. idx	lr cand.	lr util.	same θ_T	same steps
stationary	r0	[4, 3]	[4, 3]	0.1	0.1	yes	yes
stationary	r1	[1, 5]	[0, 2]	2	0.05	no	no
stationary	r2	[3, 5]	[3, 5]	2	2	yes	yes
drift	r0	[4, 3]	[4, 3]	2	2	yes	yes
drift	r1	[1, 5]	[0, 2]	2	0.2	no	no
drift	r2	[3, 5]	[3, 5]	2	2	yes	yes
reshuffle	r0	[4, 3]	[4, 3]	0.1	0.1	yes	yes
reshuffle	r1	[1, 5]	[0, 2]	2	0.05	no	no
reshuffle	r2	[3, 5]	[3, 5]	2	2	yes	yes

control counts as explained away only if the candidate beats it with the same sign in all three seeds; no control met that criterion.

Why the candidate and the utility-only control coincide in two seeds (Table 3). The candidate’s greedy early stop selected rank 2 in every condition and seed. The utility-only control is rank-matched and reuses the same pool and meta statistics, so it selects the top two directions by supervised gain. In seeds r0 and r2 these were exactly the directions the penalized score kept, so the two arms used the same projector $P_S P_S^\top$. Development tuning then selected the same learning rate for both, because tuning sees identical computations. Everything else is shared by construction: the source model and θ_0 (checked before every run), the batches and their order, and a deterministic update without randomness, momentum or buffers. The two runs are therefore the same computation. The raw logs confirm identical terminal parameters and identical per-step update norms on all orders in 6 of the 9 condition–seed pairs. The order penalty altered the selected subspace only in seed r1, where the candidate kept pool directions [1, 5] instead of [0, 2], development tuning selected a larger learning rate, and the candidate was worse by +3.60 pp. We do not read efficacy into any single seed. The observation is that, in this implementation, the penalty rarely changed the selection, and when it did the change did not help.

Drift (Table 4). Scored on the current regime’s holdout at each block end, no adapting arm improved on no adaptation by the MIE in any seed. The candidate’s seed-mean difference from the utility-only control was +0.39 pp for the current regime (within the tolerance of 0.50 pp) and +0.63 pp for online error (outside it). Its gain over no adaptation was negative on both metrics. On-line errors and the mixture-holdout errors are in Tables 6 and 7. Drift results are reported apart from stationary results and are not pooled with them.

Secondary condition and diagnostics. Re-forming batches for every order (Table 5) gave the same qualitative pattern: the candidate-minus-utility-only difference was +0.00, +3.63 and +0.00 pp.

Table 4: Drift condition (single-domain batches fixed; orders permute the domain-block sequence and batches within blocks). Err: mean over the four block ends of the current-regime holdout error (%); SD over orders (pp); Gain vs. no adaptation (pp).

Arm	World seed r0			World seed r1			World seed r2		
	Err	SD	Gain	Err	SD	Gain	Err	SD	Gain
No adaptation	20.83	0.00	+0.00	18.17	0.00	+0.00	36.83	0.00	+0.00
EM-toy (full affine)	21.25	1.33	−0.42	18.50	1.51	−0.33	37.23	0.68	−0.40
EM-toy, lr $\times 0.3$	21.67	0.45	−0.83	17.42	0.66	+0.75	36.88	0.39	−0.04
EM-toy, lr $\times 0.1$	21.52	0.21	−0.69	18.58	0.33	−0.42	36.81	0.69	+0.02
Order-penalized subspace (candidate)	21.79	1.95	−0.96	19.19	0.63	−1.02	44.90	3.50	−8.06
Utility-only subspace	21.79	1.95	−0.96	18.02	0.69	+0.15	44.90	3.50	−8.06
Per-step norm-matched (causal)	22.10	1.51	−1.27	18.15	8.87	+0.02	40.04	0.95	−3.21
Global norm-matched (calibrated)	20.67	0.82	+0.17	24.38	10.24	−6.21	37.02	0.39	−0.19

The candidate’s tuned learning rate was the largest grid value, 2, in 7 of 9 condition–seed pairs. We therefore cannot claim that its learning rate was well tuned, and we did not widen the grid.

Predicted classes on the holdout became more concentrated in seed r2. The largest class share was 0.78 for the candidate against 0.59 without adaptation, and 0.86 in drift. This is partial concentration, not a single-class collapse (Table 9).

Actual per-arm spend, including the lockstep reference of the causal control and the shared meta statistics of the utility-only control, is listed in Table 10.

6 DISCUSSION AND LIMITATIONS

What the result does and does not say. Under matched meta supervision, adaptation budget and batch membership, the order penalty did not add utility in this synthetic implementation. Its only observable effect was to change the selected subspace in one of three world seeds, and there the change hurt. This non-support is not evidence that the penalty has zero effect, and it is not evidence that it is equivalent to the controls. It says nothing about order regularization in general or about image-based TTA.

The baseline had no adaptation gain. The EM-toy baseline was worse than no adaptation on the stationary terminal holdout in every seed, and no arm reached the gain MIE. Our hypothesis concerns *retaining* adaptation gain while reducing order sensitivity. An environment in which adaptation does not help cannot evaluate that hypothesis adequately. This is a design limitation of the environment, not the outcome of a formal power analysis, which we did not perform. Choosing an environment in which the baseline adapts usefully is left as planned work (Appendix A).

Discrete pool and greedy selection. The candidate chooses among 2×4 fixed orthonormal pool directions with a greedy, early-stopping rule. With rank 2 selected everywhere, the space of distinct choices was small, which makes coincidence with the utility-only control likely. A continuous or larger pool could behave differently; we did not test one.

Learning-rate grid boundary. The candidate’s development-selected learning rate was the grid maximum in 7 of 9 condition–seed pairs. Selecting the upper boundary does not show that the optimum was found, and all learning-rate-dependent comparisons inherit this caveat. We kept the grid fixed rather than widening it after seeing results.

Few worlds and sampled orders. There are 3 world seeds and 8 sampled orders per seed. The worst observed order is not a bound on the risk over all orders. We report per-seed paired differences instead of confidence intervals over pooled orders.

Supervised meta access. The candidate and the utility-only control are fitted with labelled meta data under simulated shifts, and development tuning reads development-holdout labels for every

tuned arm. None of the adapting arms reads test labels, but the subspace arms are not fully source-free, and the meta shifts are only assumed to resemble the test shifts.

Model and optimizer scope. The model is linear with 2×4 adapted parameters, normalization uses fixed source statistics, and the optimizer is plain SGD with a constant learning rate. The analysis in Section 4 is local, and we make no statement about Adam, long horizons or batch-statistics dynamics.

7 CONCLUSION

We asked whether an order penalty adds utility to test-time adaptation when meta supervision, adaptation budget and batch membership are held fixed. We tested this in a synthetic, fully auditable model. The model includes a utility-only ablation, reduced learning rates, and causal and calibrated update-norm matching. The pre-registered rule did not support an added utility. In two of three world seeds the penalty did not change the selected projector, so the penalized and unpenalized runs were identical computations; in the third seed it changed the projector for the worse. Because the entropy baseline itself did not improve on no adaptation, this environment cannot adequately test whether order robustness can be gained without losing adaptation gain. We keep the candidate on hold. We report the protocol and the negative diagnostic as they are, without extending them to real-image TTA.

AI USE STATEMENT

Generative AI (Claude Code, an AI coding agent, operating in a repository session) was used for tasks that require disclosure, as follows.

- It designed and implemented all code, tests and configurations, and ran the experiments.
- It proposed the candidate method and the pre-registered decision rule.
- It drafted this manuscript and generated its tables and numbers with a script that reads the raw run files.
- An AI sub-agent located and read the cited papers for the prior-art comparison. Two of those metadata records were rechecked by the main agent on arXiv.

Human review status: pending. No human has yet reviewed the code, the analyses, the citations or this text, so the claims and citations should be treated as unverified by humans.

REPRODUCIBILITY STATEMENT

The code uses only the Python standard library. The experiment is reproduced at commit 0119a34 with

```
PYTHONPATH=src python3 -m ordertta.stage
--stageconfigs/stage2_order_penalty_diagnostic.json
```

The frozen condition configurations, the stage manifest (code, configuration, data, model and hold-out hashes, CPU time and memory) and the raw per-step logs are included with the code. The script `paper/scripts/export_results.py` regenerates all tables and in-text numbers from those raw files. Appendix E lists the files and the deviations from earlier plans. Synthetic data are generated from the listed seeds; no external data or pretrained weights are used.

REFERENCES

- A. Bartler, A. Bühler, F. Wiewel, M. Döbler, and B. Yang. MT3: Meta test-time training for self-supervised test-time adaption. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2022. arXiv:2103.16201.
- M. Döbler, R. A. Marsden, and B. Yang. Robust mean teacher for continual and gradual test-time adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

- J. Gao, J. Zhang, X. Liu, T. Darrell, E. Shelhamer, and D. Wang. Back to the source: Diffusion-driven adaptation to test-time corruption. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv:2207.03442.
- T. Gong, J. Jeong, T. Kim, Y. Kim, J. Shin, and S.-J. Lee. NOTE: Robust continual test-time adaptation against temporal correlation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.05117.
- G. Gur-Ari, D. A. Roberts, and E. Dyer. Gradient descent happens in a tiny subspace, 2018. arXiv:1812.04754.
- T.-H. Hoang, D. M. Vo, and M. N. Do. Persistent test-time adaptation in recurring testing scenarios. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2311.18193.
- C. Li, H. Farkhoor, R. Liu, and J. Yosinski. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1804.08838.
- Taejun Lim, Joong-Won Hwang, and Kibok Lee. When and where to reset matters for long-term test-time adaptation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.03796.
- A. Nichol, J. Achiam, and J. Schulman. On first-order meta-learning algorithms, 2018. arXiv:1803.02999.
- S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan. Efficient test-time model adaptation without forgetting. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2204.02610.
- S. Niu, J. Wu, Y. Zhang, Z. Wen, Y. Chen, P. Zhao, and M. Tan. Towards stable test-time adaptation in dynamic wild world. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2302.12400.
- O. Press, S. Schneider, M. Kümmerer, and M. Bethge. RDumb: A simple approach that questions our progress in continual test-time adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.05401.
- Y. Shi, J. Seely, P. H. S. Torr, N. Siddharth, A. Hannun, N. Usunier, and G. Synnaeve. Gradient matching for domain generalization. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2104.09937.
- I. Shumailov, Z. Shumaylov, D. Kazhdan, Y. Zhao, N. Papernot, M. A. Erdogdu, and R. Anderson. Manipulating SGD with data ordering attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2104.09667.
- S. L. Smith, B. Dherin, D. G. T. Barrett, and S. De. On the origin of implicit regularization in stochastic gradient descent. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2101.12176.
- John Sweeney. The geometry of sequential learning: Lie-bracket prediction of transfer order. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.24993.
- D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.10726.
- Q. Wang, O. Fink, L. Van Gool, and D. Dai. Continual test-time domain adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2203.13591.
- L. Yuan, B. Xie, and S. Li. Robust test-time adaptation in dynamic scenarios. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv:2303.13899.
- H. Zhao, Y. Liu, A. Alahi, and T. Lin. On pitfalls of test-time adaptation. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2306.03536.

A PLANNED EXPERIMENTS (NOT RUN)

None of the following has been run. They are listed so that the scope of the present evidence is explicit.

- **An environment in which adaptation helps (NOT RUN).** Before revisiting any order-robustness question, choose a setting in which the entropy baseline improves on no adaptation by more than the MIE on held-out data. The current synthetic worlds do not meet this condition. Selecting the new setting is a design decision that has not been taken.
- **Real-image TTA (NOT RUN).** The planned protocol is CIFAR-10-C at severity 5 with a pretrained network and Tent-style normalization adaptation. It would use the same separation of stationary fixed-batch, re-batched and drift conditions. It is blocked: no deep-learning framework, GPU, dataset copy or pretrained weights are available in the project environment, and dataset and weight licences have not been verified by a human.
- **Remaining pre-registered seeds and orders (NOT RUN).** The original pilot lists five world seeds and sixteen orders; this diagnostic used the first three seeds and the first eight orders. Running more of them on the *same* candidate is not planned, because the candidate is on hold. We will not try to rescue it with seed, λ , learning-rate or model sweeps.
- **Continuous or larger subspace pools (NOT RUN).** This would test whether the coincidence with the utility-only control is an artefact of the eight-direction pool.

B ADDITIONAL TABLES

Table 5: Secondary condition (samples reshuffled and re-batched per order). Same columns as Table 1. Not used by the decision rule.

Arm	World seed r0			World seed r1			World seed r2		
	Err	SD	Gain	Err	SD	Gain	Err	SD	Gain
No adaptation	20.83	0.00	+0.00	18.17	0.00	+0.00	36.83	0.00	+0.00
EM-toy (full affine)	23.12	1.35	−2.29	18.46	0.08	−0.29	36.98	0.11	−0.15
EM-toy, lr $\times 0.3$	22.62	0.29	−1.79	18.33	0.00	−0.17	36.44	0.12	+0.40
EM-toy, lr $\times 0.1$	21.96	0.15	−1.12	18.00	0.00	+0.17	36.46	0.08	+0.38
Order-penalized subspace (candidate)	20.50	0.00	+0.33	21.29	0.21	−3.12	44.48	2.12	−7.65
Utility-only subspace	20.50	0.00	+0.33	17.67	0.00	+0.50	44.48	2.12	−7.65
Per-step norm-matched (causal)	21.02	0.27	−0.19	17.23	1.30	+0.94	38.54	1.30	−1.71
Global norm-matched (calibrated)	20.88	0.08	−0.04	19.27	1.08	−1.10	36.38	0.08	+0.46

Table 6: Drift condition, predict-then-update online error over the stream (%), SD over orders (pp) and gain vs. no adaptation (pp).

Arm	World seed r0			World seed r1			World seed r2		
	Err	SD	Gain	Err	SD	Gain	Err	SD	Gain
No adaptation	18.33	0.00	+0.00	16.04	0.00	+0.00	35.83	0.00	+0.00
EM-toy (full affine)	19.61	1.96	−1.28	15.86	1.73	+0.18	37.68	0.85	−1.85
EM-toy, lr $\times 0.3$	18.62	0.80	−0.29	15.08	0.85	+0.96	36.38	0.65	−0.55
EM-toy, lr $\times 0.1$	18.10	0.42	+0.23	15.70	0.51	+0.34	35.55	0.22	+0.29
Order-penalized subspace (candidate)	20.03	1.87	−1.69	16.90	0.85	−0.86	44.06	3.75	−8.23
Utility-only subspace	20.03	1.87	−1.69	15.03	0.81	+1.02	44.06	3.75	−8.23
Per-step norm-matched (causal)	20.39	1.81	−2.06	15.94	7.82	+0.10	41.88	2.07	−6.04
Global norm-matched (calibrated)	18.62	0.77	−0.29	21.61	10.12	−5.57	36.56	0.69	−0.73

Table 7: Drift condition, terminal error on the mixture holdout (%). Reported for completeness only; it penalises adaptation to the latest regime and is not used by the decision rule.

Arm	World seed r0			World seed r1			World seed r2		
	Err	SD	Gain	Err	SD	Gain	Err	SD	Gain
No adaptation	20.83	0.00	+0.00	18.17	0.00	+0.00	36.83	0.00	+0.00
EM-toy (full affine)	22.10	1.40	−1.27	18.90	0.73	−0.73	37.17	0.56	−0.33
EM-toy, lr $\times 0.3$	22.12	0.50	−1.29	17.35	0.23	+0.81	36.54	0.15	+0.29
EM-toy, lr $\times 0.1$	21.77	0.12	−0.94	18.15	0.06	+0.02	36.50	0.00	+0.33
Order-penalized subspace (candidate)	24.10	1.69	−3.27	21.19	0.14	−3.02	49.31	3.65	−12.48
Utility-only subspace	24.10	1.69	−3.27	17.88	0.19	+0.29	49.31	3.65	−12.48
Per-step norm-matched (causal)	23.52	2.13	−2.69	19.02	9.80	−0.85	40.44	2.30	−3.60
Global norm-matched (calibrated)	21.21	0.78	−0.37	27.10	10.83	−8.94	36.65	0.27	+0.19

Table 8: Primary stationary condition: worst observed order (maximum terminal error over the 8 sampled orders, %). This is not a bound over all orders.

Arm	World seed r0	World seed r1	World seed r2
	Max	Max	Max
No adaptation	20.83	18.17	36.83
EM-toy (full affine)	24.00	25.33	37.17
EM-toy, lr $\times 0.3$	23.17	18.67	36.50
EM-toy, lr $\times 0.1$	22.17	17.83	36.50
Order-penalized subspace (candidate)	20.67	21.67	45.83
Utility-only subspace	20.67	17.67	45.83
Per-step norm-matched (causal)	20.83	21.00	40.33
Global norm-matched (calibrated)	21.00	27.50	36.50

Table 9: Primary stationary condition diagnostics per world seed: selected learning rate, mean update path length $\sum_t \|\Delta\theta_t\|$ over orders, and the largest predicted-class share on the holdout over orders (collapse indicator; 1/3 is balanced for 3 classes).

Arm	World seed r0			World seed r1			World seed r2		
	lr	path	max share	lr	path	max share	lr	path	max share
No adaptation	0	0.00	0.37	0	0.00	0.45	0	0.00	0.59
EM-toy	1	3.70	0.53	1	4.39	0.54	0.5	1.99	0.69
EM-toy lr $\times 0.3$	0.3	1.40	0.43	0.3	1.69	0.46	0.15	0.93	0.68
EM-toy lr $\times 0.1$	0.1	0.55	0.37	0.1	0.65	0.45	0.05	0.45	0.66
Candidate	0.1	0.24	0.37	2	2.98	0.43	2	4.33	0.78
Utility-only	0.1	0.24	0.37	0.05	0.29	0.45	2	4.33	0.78
Norm-matched (step)	0.1	0.24	0.37	2	2.98	0.49	2	4.33	0.72
Norm-matched (global)	0.0348	0.21	0.37	1.21	5.15	0.56	0.166	0.99	0.68

Table 10: Actual spend per arm in the primary stationary condition, summed over 3 world seeds: gradient evaluations (finite-difference Hessian-vector products counted as their two gradient calls; forward-only Jacobian-vector products not counted) and wall-clock seconds by phase. Utility-only reuses the candidate’s meta statistics; its meta-fit cost is selection only. The causal per-step control includes the lockstep reference replica. Source training (shared by all arms) is listed once.

Arm	Meta fit		Tuning / calib.		Test time	
	evals	s	evals	s	evals	s
No adaptation	0	0.00	0	0.00	0	1.14
EM-toy	0	0.00	576	1.40	720	1.89
EM-toy lr $\times 0.3$	0	0.00	0	0.00	720	2.06
EM-toy lr $\times 0.1$	0	0.00	0	0.00	720	1.93
Candidate	432	3.39	576	1.49	720	1.93
Utility-only	0	1.31	576	1.48	720	2.00
Norm-matched (step)	0	0.00	0	0.00	1440	3.04
Norm-matched (global)	0	0.00	1120	2.09	720	2.09
Source training (shared)	600 full-batch epochs, 10.41 s					

C DERIVATION DETAILS AND NUMERICAL CHECKS

Sign convention. $U_b(U_a(\theta))$ applies a first. In the code, `two_step_order_difference(theta, a, b)` returns $\theta_{ab} - \theta_{ba}$ with $\theta_{ab} = U_b(U_a(\theta))$, which matches Eq. (2).

Remainder. Eq. (3) requires only that each Hessian be Lipschitz on the relevant segment. It follows from the integral form of Taylor’s theorem applied to ∇L_b along $[\theta, \theta - \eta g_a]$, and symmetrically for ∇L_a . This is a standard estimate, not a new result. For K steps, $\theta_K = \theta - \eta \sum_k g_{\pi_k} + \eta^2 \sum_{i < j} H_{\pi_j} g_{\pi_i} + O(\eta^3)$, again under local regularity.

Numerical fixtures (implementation checks, not proofs). The repository tests (`tests/test_quadratic_order.py`) cover five cases:

1. On random quadratics, the exact two-step difference matches the second-order term to floating-point precision for $\eta \in \{10^{-3}, 0.05, 0.3, 1\}$.
2. On a quartic-perturbed quadratic, the remainder shrinks at an empirical order between 2.8 and 3.2 when η is expressed in units of the local curvature L .
3. On the same quartic fixture, the relative remainder is below 0.05 at $\eta L = 10^{-3}$ and exceeds 0.1 at $\eta L \approx 0.3$.
4. A three- and four-step prediction has a remainder of order three.
5. Only plain SGD is accepted, and stateful optimizers are rejected.

The exact-rational and remainder-bound checks run for this draft are in Appendix F.

D INFORMATION ACCESS AND IMPLEMENTATION DETAILS

Table 11: Information used by each arm. No arm reads test-stream or test-holdout labels. Test-stream labels are read only by a predict-then-score oracle after the prediction for that sample is committed, and holdout labels are read only by the evaluators.

Arm	Meta data and labels	η source	Extra information
No adaptation	–	–	–
EM-toy (full affine)	–	development grid	–
EM-toy, lr $\times$ 0.3 / $\times$ 0.1	–	EM-toy η times factor	–
Candidate	yes	development grid	–
Utility-only	same as candidate	development grid	candidate’s realised rank
Per-step	–	direction only	candidate’s step- t norm
norm-matched	–	–	(lockstep replica)
Global	–	calibration	candidate’s mean terminal
norm-matched	–	bisection	displacement

Development tuning reads development-holdout labels for all tuned arms, and this is recorded in the cost ledger. The calibration of the global control reads no labels. The causal per-step control computes the candidate’s update on the same batch at the same step, so it cannot read future steps. Its replica’s gradient cost is charged to the control, and a post-hoc audit found the replica’s per-step norms equal to the recorded candidate run on every order.

E REPRODUCIBILITY DETAILS AND DEVIATIONS

- **Frozen configuration.** `configs/stage2_order_penalty_diagnostic.json` (stage and decision rule),
`configs/stage2_stationary_fixed_batches.json`,

configs/stage2_drift_fixed_batches.json and
configs/stage2_stationary_resuffle.json. All were committed before
the run.

- **Raw outputs.** runs/stage2_order_penalty_0119a34/, containing the stage summary and manifest, and for each condition raw_log.jsonl, summary.json and manifest.json.
- **Design changes before this run.** An earlier plan (two toy pilot configurations with re-formed batches and a pooled-bootstrap decision rule) was never run. Before the run we made the following changes:
 - added the utility-only control;
 - fixed batch membership for the primary comparisons;
 - made per-step norm matching causal and removed label reads from calibration;
 - replaced the pooled bootstrap with per-seed paired differences.
 A separate code-path smoke run on a smaller world is recorded in runs/cpu_smoke_b7edd9f/. It predates these changes and is not evidence.
- **Display names.** Arm identifiers in logs and configurations are unchanged; for example, tent_full is displayed as EM-toy.

F FORMULA CHECKS FOR THIS DRAFT

Two small checks were run with paper/scripts/check_two_step_identity.py (seed fixed in the script; output paper/generated/formula_check.json). Both are numerical checks of a standard Taylor estimate, not proofs.

- **Exact quadratic identity.** In 200 random cases with rational symmetric Hessians, rational centres, starting points and step sizes, in dimension 2–4, the rational arithmetic difference $U_b(U_a(\theta)) - U_a(U_b(\theta))$ equalled $\eta^2(H_b g_a - H_a g_b)$ exactly in all cases.
- **Remainder bound.** We used quartic-perturbed quadratics $L(x) = \frac{1}{2}r^\top A r + \frac{\kappa}{4} \sum_i r_i^4$ with $r = x - c$. On any segment where $|r_i| \leq R$, $6\kappa R$ is a valid Lipschitz constant of the Hessian in operator 2-norm. With this constant, the bound of Eq. (3) held in 200 of 200 random cases with $\eta \in [10^{-3}, 10^{-0.5}]$. The largest remainder-to-bound ratio was 0.996, so the bound is not vacuous on these fixtures.